CS/EE 120B Custom Laboratory Project Report

TurretLock

Pranae Harirajan

06/08/2026

Introduction:

TurretLock is a real-time targeting game. A 4-in-1 LED Matrix (32x8) pixels is the playfield: solid walls sit on the far-left and far-right four columns, and a small 2x2 "target" appears in the open space between them. The player moves a single aim dot with an analog joystick and tries to line it up with the target. When the dot touches the target, a green LED lights to indicate a lock; otherwise, a red LED blinks. Pressing the joystick button fires: a shot on the target scores 100 points and spawns a new target, while a shot taken off-target costs one life. A pair of servo motors act as a physical turret that swings to follow the aim.

The player starts with five lives. Targets are endless, so the score can grow without limit. When all five lives are lost, the game ends, and the screen displays the final score along with the all-time high score, which is stored in EEPROM and survives power cycles. A small color TFT screen acts as a heads-up display for the score, the remaining lives, which are drawn as heart sprites, and the game messages: start screen, game over, and a high-score reset prompt.

I was able to fully and completely finish the game as per the proposal as much as possible, with slight additions and removals. This includes full core gameplay, two-axis servo turret, lock LEDs(green and red), score, five-life system with heart sprites, endless targets, EEPROM high score(referred to as “Hi” on the TFT), start/game-over/reset screens, and a click-to-reset gesture for the high score. The whole program is built as Concurrent SynchSMs driven by a task scheduler.

Some of the limitations I had were that the joystick is spring-loaded, so the aim is velocity-based(push to move, and release to hold), so I had to update the location to avoid the velocity issue and the rubberbanding of the joystick positioning. This was difficult to clear and was hard to create the absolute position.

Elements of Complexity: (All below have been implemented)

- ➔ EEPROM-backed high score (added): The all-time high score is read from EEPROM at power-up and written back whenever it is beaten, so it persists across power cycles. A single click on the game-over screen opens a confirmation prompt to wipe it.
- ➔ Custom AVR ST7735 TFT driver: Since we were not able to use libraries (I wasn't sure if I could), I wrote a driver that brings up the color TFT (reset, sleep-out, gamma, 16-bit color) and draws filled rectangles, a 5x7 text font, centered text, and a heart sprite. It shares the SPI data/clock lines with the LED Matrix using separate chip-select lines.
- ➔ Custom AVR MAX7219 Matrix Driver: I also wrote a bit-select driver that drives 4 daisy-chained 8x8 panels(soldered as part of the matrix display) and periodically re-sends the configuration registers so a panel glitched by electrical noise self-heals.
- ➔ Two-Axis Servo Turret Aim: There was one servo as part of the proposal, but I updated it to two servos. Each servo correlated to the x and y direction, to track in the 2D coordinate system. Using Timer1 in the background, the motion stays smooth while the game runs.
- ➔ Click to Reset Gesture: On the game-over screen, a single click opens a confirmation prompt to safely reset the high score, and a double click starts a new game/restarts the game.

User Guide:

Controls:

- ➔ Joystick (X/Y): moves the aim dot on the matrix. The stick is a speed control
  - ◆ Push to move the dot, release to leave it in place.
- ➔ Joystick button - single click:
  - ◆ On the Start screen, a game begins.
  - ◆ During play: fires at the current aim position.
  - ◆ On the Game Over screen: opens the "Reset High Score?" prompt.
  - ◆ On the Reset prompt: confirms wiping the high score.
- ➔ Joystick button - double click (Game Over screen): starts a new game.

Visual/Physical Outputs:

- ➔ **LED Matrix:** the playfield - walls, the 2x2 target, and the aim dot.
- ➔ **TFT Screen:** start screen, live score and hearts (lives) during play, and the game-over / reset screens.
- ➔ **Red LED:** blinks while a target is not yet locked. Green LED: solid while the aim is on the target.
- ➔ **Servos:** swing to match the aim, like a turret.

Normal flow: Power On → Start screen (shows recorded high score) → 1 click to play → score by hitting targets, lose a life on each miss (5 starting lives) → at 0 lives the Game Over screen shows the final and high scores → 1 click to reset the high score (Reset score phase), then one click resets the high score or waiting ignores this phase, regardless of the 1 click phase → Start Game Phase, Double click at Game Over Phase goes to restart the game, but doesnt remove the previously recorded High Score.

Hardware Components Used:

- ➔ Arduino Uno R3 (ATmega328P microcontroller)
- ➔ MAX7219-based 4-in-1 8x8 LED matrix module (32x8 display)
- ➔ ST7735 1.44" SPI color TFT LCD (128x128) (Curr. Lim. Res. for Backlight, 100 ohms)
- ➔ Analog joystick module (X, Y, and push-button → built within the joystick)
- ➔ 2x hobby servo motors (turret X and Y coordinates)
- ➔ 2x LEDs (red, green) with current-limiting resistors (220 ohms)
- ➔ Breadboard, jumper wires, and a separate 5V supply for the servos (common GND)

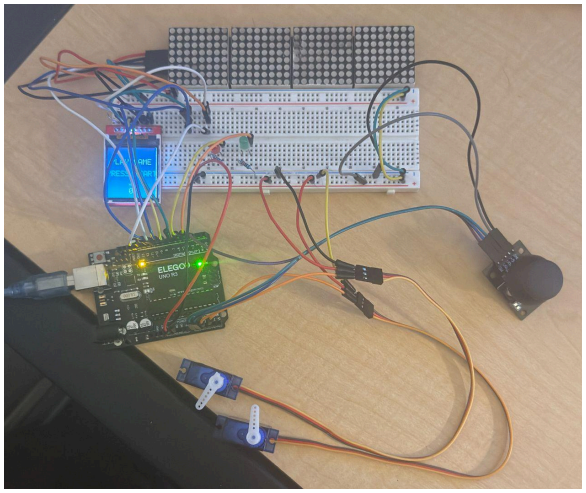
Software Libraries Used: *(No external/3rd-party libraries used, only AVR/toolchain headers)*
(All peripheral drivers (LED matrix, TFT, ADC/joystick) written from scratch, as referred above)

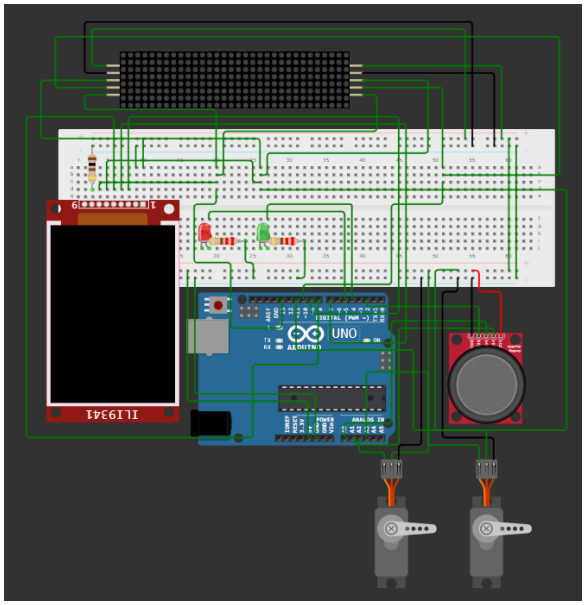
- ➔ avr/io.h - register and pin definitions for the ATmega328P.
- ➔ avr/interrupt.h - the ISR() macro and interrupt control, used for the Timer1 servo pulses and the Timer2 scheduler tick.
- ➔ avr/eeprom.h - eeprom_read_word / eeprom_update_word to persist the high score.
- ➔ util/delay.h - short blocking delays used only in the TFT power-on sequence.
- ➔ avr/pgmspace.h - stores the font and heart bitmaps in flash (PROGMEM) to save RAM.

Pin Map Table:

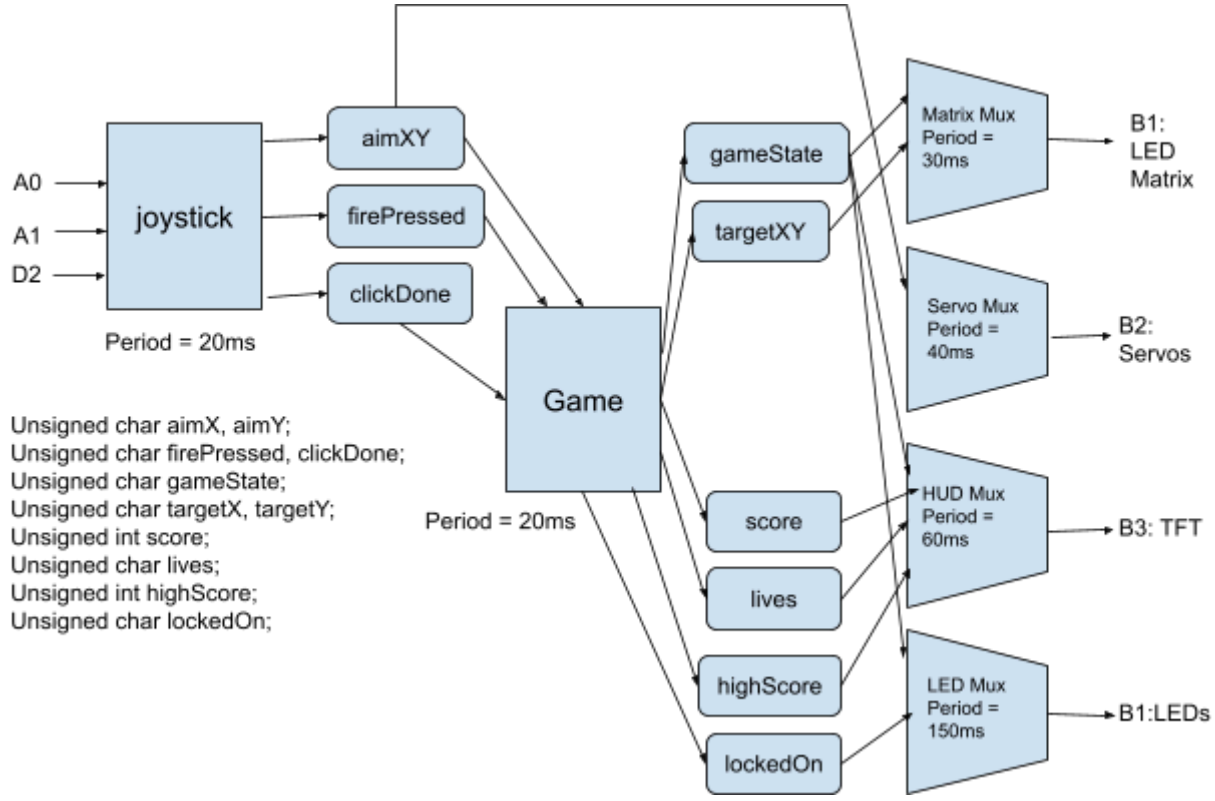
Arduino Pin	Connected To	Direction
A0/A1	Joystick VRx/VRy	IN
A2/A3	Servo X/Y Signal	OUT
D2	Joystick Button SW	IN
D4/D5	LED: Green/RED (220 ohms)	OUT
D7	TFT RST	OUT
D8	TFT DC	OUT
D9	TFT CS	OUT
D10	MAX7219 CS	OUT
D11	MAX7219 DIN + TFT SDA (shared MOSI)	OUT
D13	MAX7219 CLK + TFT SCK (shared SCK)	OUT
5V/GND	Full System Power + Arduino UNO R3	------

Wiring Diagram: (Schematic + Physical) →



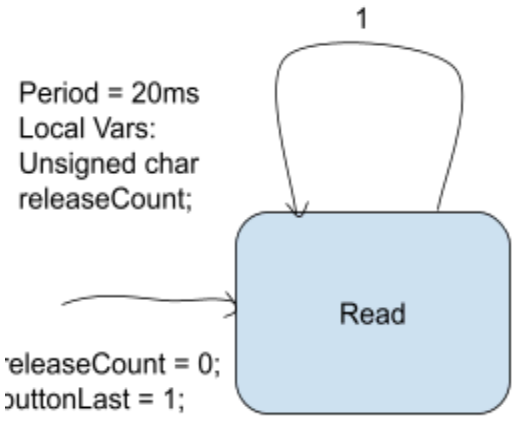


Task Diagram:



SynchSMs:

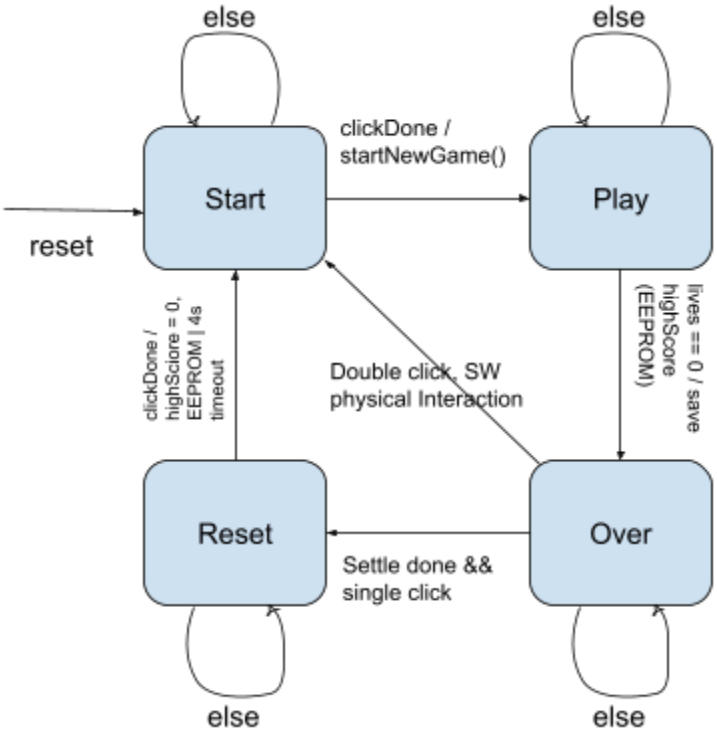
Task 1 - Joystick:




```
joyX = ADC(0); joyY = ADC(1)
pushX = joyX - 512; pushY = joyY - 512;
If (pushX > -D2 && pushX < DZ) { pushX = 0; }
If (pushY > -D2 && pushY < DZ) { pushY = 0; }
fineX += pushX / JOY_SPEED;
fineX += pushX / JOY_SPEED;
clamp(fineX, 0, (RIGHT_EDGE - LEFT_EDGE)*FINE);
clamp(fineY, 0, (ROWS - 1)*FINE);
aimX = LEFT_EDGE + fineX / FINE;
aimY = fineY / FINE;
B = (PIND >> BUTTON) & 1;
If (b == 0) {
        If (buttonLast == 1) {
                firePressed = 1;
                buttonLast = 0;
                releaseCount = 0;
        }
} else if (buttonLast ==  0) {
        If (++releaseCount >= 2) {
                clickDone = 1;
                buttonLast = 1;
        }
}
```

Task 2 - Game Task:

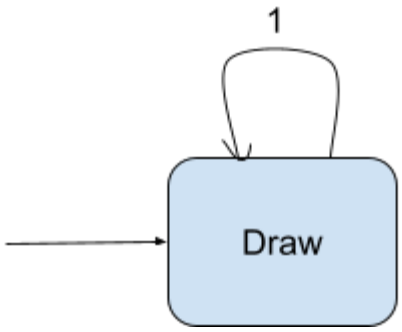
Period = 20ms
Unsigned int resetTimer, overSettle,
overClickTimer;
Unsigned char overClicks;




```
Per-Tick state action
(firePressed reset each tick):
Play: lockedOn = checkLock();
        If (firePressed) {
                If (lockedOn) {
                        score += 100;
                        newTarget();
                } else if (--lives == 0) {
                        state = Over;
                }
Over: if (overSettle < 600) {
        overSettle += 20;
       } else {
        // count taps over DBL_WINDOW
        // → 1=RESET, 2=START
        break;
       }
Reset:
resetTimer += 20;
show Disp "RESET HI SCORE?"
```

Task 3 - Matrix Task:

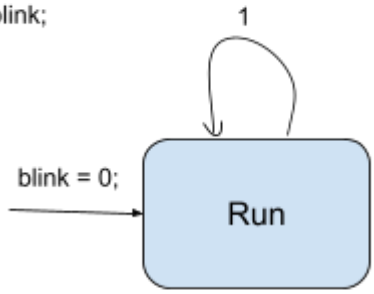
Period = 30ms
Unsigned char grid[8][4];




```
for (r, p) grid[r][p] =  0;
If (gameState == ST_PLAY) {
        for (y = 0; y < ROWS; y++){
                for (w = 0; w < WALL_W; w++){
                        plot(grid, w, y);
                        plot(grid, COLS-1-w, y);
                }
        plot(grid, targetX, targetY);
        plot(grid, targetX+1, targetY);
        plot(grid, targetX, targetY+1);
        plot(grid, targetX+1, targetY+1);
        plot(grid, aimX, aimY);
}
matrixDraw(grid);
```

Task 4 - LED Task:

Period = 40ms
Unsigned char
blink;

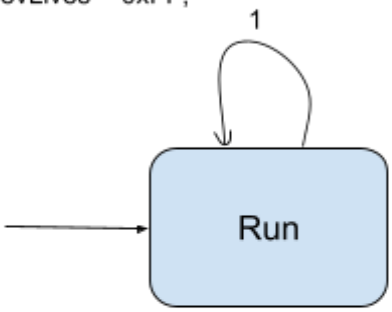



```
blink ^= 1;
If (gameState == ST_PLAY) {
    If (lockedOn) {
        GREEN_LED =  1;
        RED_LED = 0;
    } else {
        GREEN_LED  = 0;
        RED_LED = blink;
    }
} else {
    GREEN_LED = 0;
    RED_LED = 0;
}
```

Task 5 - HUD/TFT Task:

Period = 60ms
Unsigned char prevState,
prevLives;
Unsigned int prevScore;

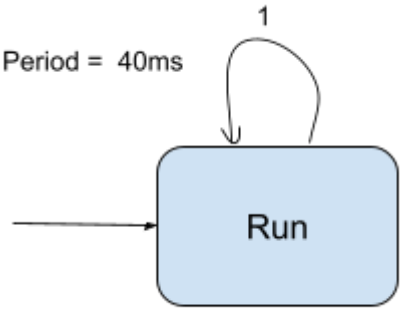
```
prevState = 0xFF;
prevScore = 0xFFFF;
prevLives = 0xFF;
```




```
If (gameState != prevState) {
        drawScreen(gameState);
        prevState = gameState;
}
If(gameState == ST_PLAY) {
        If (score != prevScore) {
                drawScore(score);
                prevScore = score;
        }
        If (lives !=  prevLives) {
                drawHearts(lives);
                prevLives = lives;
        }
}
```

Task 6 - Servo Task:

Period = 40ms




```
OCR1A = map_value(LEFT_EDGE,
RIGHT_EDGE, SERVO_LEFT,
SERVO_RIGHT, aimX);
OCR1B = map_value(0, ROWS-1,
SERVO_LEFT, SERVO_RIGHT, aimY);
```